Article 2

Title: PointerGuess: Targeted Password Guessing Model Using Pointer Mechanism

Authors: Kedong Xiu and Ding Wang, *Nankai University*

Published: 33rd USENIX Security Symposium

Year: 2024

Link: https://www.usenix.org/conference/usenixsecurity24/presentation/xiu

Summary

In today's day and age, we most commonly use textual passwords to manage our account access across dozens of websites. This becomes a big target for malicious agents and more often than not they end up succeeding because of the simple fact that we reuse or slightly change the already in use passwords rather than making new ones. This is a major security risk. There are guessing tools available at the disposal of attackers that explore how people edit an older password by modifying it. It can take an existing password then insert, delete or substitute any character(s) in the password to figure out any modified passwords. However, it becomes increasingly difficult as the password becomes more complex.

In the paper, authors Xiu and Wang introduce PointerGuess which is classified as a guessing model that uses a new pointer mechanism that points to each character and directly decides where or not to copy part of the old password or generate a new character. This allows PointerGuess to handle small changes in the password as well as any big structural alterations without needing numerous special editing operations.

Under the hood of PointerGuess, we see a very interesting system in place:

1. Seq2Seq architecture: In essence, this is a standard 'sequence-to-sequence' neural network which is a type of neural network that maps input sequences of different lengths to output sequences of different lengths. So in this case it could read a user's old password (e.g. "IloveMacOP") and tries to predict the new modified on based on this (e.g. "MacOP6789") [the examples were given in the paper].
2. Pointer module: In parallel to the probabilities for each possible character in the distribution, PointerGuess would calculate a copied distribution over characters from the old, unmodified password. Then a "soft switch" would then weigh the two to compute the final probability for each of the next characters.
3. No Atomic Edits required: Since PointerGuess has learning capabilities, it can learn when to replicate existing characters and when it needs to insert a new one. This is different to older models, which require a lot of definitions and rules, saving resources. Due to the learning capacity of PointerGuess, it is enabled to capture longer-range reuse cases (paper gives the example of moving "MacOP" from the front side of the password to the backside).

The professionals conducted two extensive scenario tests by using 13 real-world breached datasets, which contained roughly over 4.8 billion passwords and found that:

- Single-Source Guessing (Standalone PointerGuess): When the attacker has one of the old passwords, PointerGuess was able to use it to crack 25.21% of different targets from 100 guesses for average users and still compared better to the best model prior by a staggering 38% on average.
- Multi-Source Guessing (MS-PointerGuess): When the attacker has multiple leaked passwords then the success rate of guessing rose to another 31% on average per 100 guesses.

The professionals then analyzed it across a dozen realistic “attack scenarios” which involved matching users across different sites and found that PointerGuess performed better than any leading methods such as ‘Pass2Path’ and ‘Pass2Edit’ (which are also similar models). PointerGuess outperformed them in 10 of the 12 cases tested before even introducing popular password dictionaries. One more major plus for PointerGuess is that it is not resource intensive as the model is lightweight (approximately 2.3MB) and guesses passwords fast (around 9,700 passwords/seconds), which are specs that make it really efficient for real-world testing.

In conclusion we can surely say that PointerGuess represents a massive leap forward in the understanding of the behavior of people when it comes to reusing passwords. By directly modelling the decision to copy an older password than to create a new one, it:

- Captures the complex reuse of passwords: PointerGuess handles any bulk segment manipulation and email-suffixes copying that could cause errors or be missed by editing-based models.
- Improves personalized guessing techniques: Attackers (or cybersecurity testers) can get an accurate assessment of likely password changes despite the guess limits recommended by the National Institute of Standards and Technology.
- Enable stronger defense mechanisms: The professionals discuss the idea of a “password-reuse strength meter” which centers around the idea to flag accounts which are at higher risk based on PointerGuess’s assessment of password protection.

Xiu and Wang’s work clearly highlights that the pointer-based model generation is a formidable tool for assessing password security and driving future protections against smart attacks involving reused passwords.